

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

Coding Challenge

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Coding Challenge

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.
2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.
3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.
4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.
5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

RUBRICS FOR CALCULATION:

Q. No	Understanding of Concepts (25)	Experimental Design (30)	Use of Tools (20)	Analysis of Results (15)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

CO4AT1 CRYPTOGRAPHY:

R. Deepasri (192511309)

1. IPSec AH – MAC Generation and Verification

The program creates a MAC for the IP packet and checks it again. If both MAC values are the same, the packet is **not modified**.

```
import hmac
```

```
import hashlib
```

```
key = input("Enter secret key: ").encode()
```

```
message = input("Enter IP packet data: ").encode()
```

```
mac = hmac.new(key, message, hashlib.sha256).hexdigest()
```

```
print("Generated MAC:", mac)
```

```
received = input("Enter MAC for verification: ")
```

```
if hmac.compare_digest(mac, received):
```

```
    print("Packet is authentic and integrity verified")
```

```
else:
```

```
    print("Packet is modified or authentication failed")
```

```
1 import hmac
2 import hashlib
3
4 key = input("Enter secret key: ").encode()
5 message = input("Enter IP packet data: ").encode()
6
7 mac = hmac.new(key, message, hashlib.sha256).hexdigest()
8 print("Generated MAC:", mac)
9
10 received = input("Enter MAC for verification: ")
11
12 if hmac.compare_digest(mac, received):
13     print("Packet is authentic and integrity verified")
14 else:
15     print("Packet is modified or authentication failed")
```

Output

```
Enter secret key: abc123
Enter IP packet data: 192.168.1.10 DATA
Generated MAC:
8bc001c6f8e8bf82283dafc6def7cb67ca806b8bf80e61d82d1ad306c3875188
Enter MAC for verification: 8e.....
Packet is modified or authentication failed

=== Code Execution Successful ===
```

2. IPSec ESP – Encryption and Decryption

The program encrypts the IP packet payload using a key and then decrypts it using the same key to get the **original message**.

```
def encrypt(data, key):
```

```
    return "".join(chr(ord(c) ^ ord(key[i % len(key)])) for i, c in enumerate(data))
```

```
def decrypt(data, key):
```

```
    return "".join(chr(ord(c) ^ ord(key[i % len(key)])) for i, c in enumerate(data))
```

```
data = input("Enter IP packet payload: ")
```

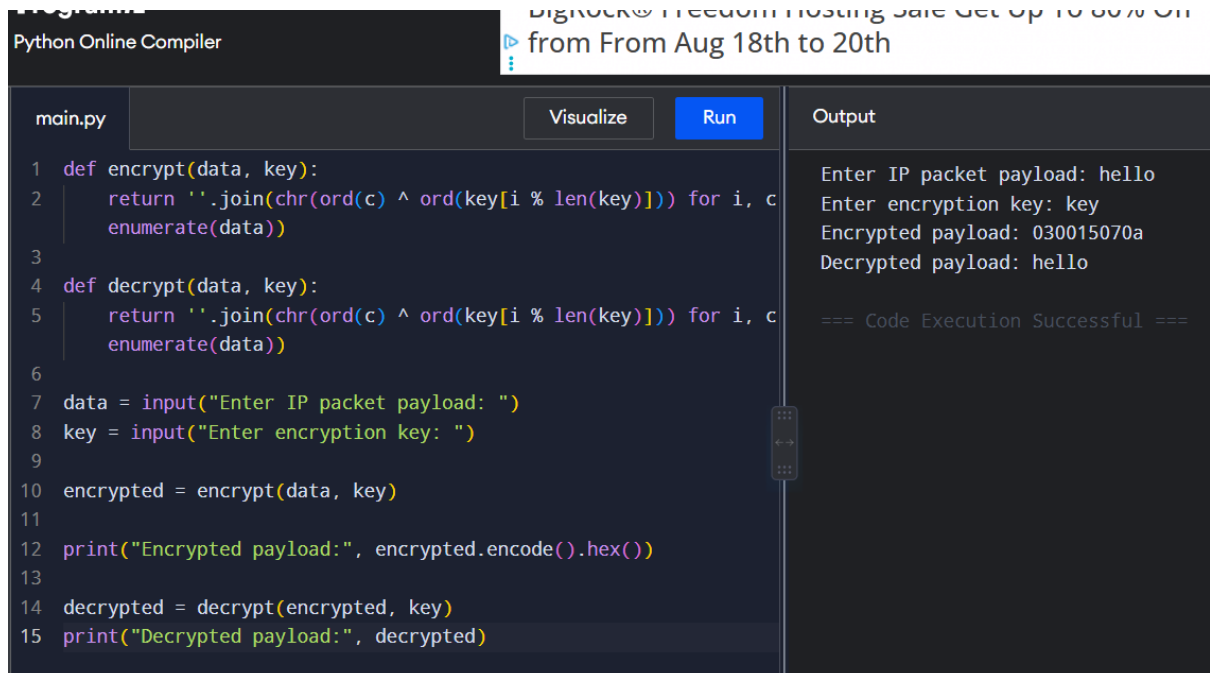
```
key = input("Enter encryption key: ")
```

```
encrypted = encrypt(data, key)
```

```
print("Encrypted payload:", encrypted.encode().hex())
```

```
decrypted = decrypt(encrypted, key)

print("Decrypted payload:", decrypted)
```



The screenshot shows a Python Online Compiler interface. The code in the editor is as follows:

```
1 def encrypt(data, key):
2     return ''.join(chr(ord(c) ^ ord(key[i % len(key)])) for i, c
3     enumerate(data))
4
5 def decrypt(data, key):
6     return ''.join(chr(ord(c) ^ ord(key[i % len(key)])) for i, c
7     enumerate(data))
8
9 data = input("Enter IP packet payload: ")
10 key = input("Enter encryption key: ")
11
12 encrypted = encrypt(data, key)
13
14 print("Encrypted payload:", encrypted.encode().hex())
15
16 decrypted = decrypt(encrypted, key)
17 print("Decrypted payload:", decrypted)
```

The output window shows the following interaction:

```
Enter IP packet payload: hello
Enter encryption key: key
Encrypted payload: 030015070a
Decrypted payload: hello

=== Code Execution Successful ===
```

3. IPSec Security Association (SA) Management

The program allows the user to **create, search, and delete** IPSec Security Associations.

```
SA = {}
```

```
while True:
```

```
    print("\n1.Create 2.Lookup 3.Delete 4.Display 5.Exit")
```

```
    choice = input("Enter choice: ")
```

```
    if choice == "1":
```

```
        spi = input("Enter SPI: ")
```

```
        src = input("Enter source IP: ")
```

```
        dst = input("Enter destination IP: ")
```

```
SA[spi] = (src, dst)

print("Security Association created")
```

```
elif choice == "2":

    spi = input("Enter SPI: ")

    if spi in SA:

        print("SA found:", SA[spi])

    else:

        print("SA not found")
```

```
elif choice == "3":

    spi = input("Enter SPI: ")

    if spi in SA:

        del SA[spi]

        print("Security Association deleted")

    else:

        print("SA not found")
```

```
elif choice == "4":

    print("Security Associations:", SA)
```

```
elif choice == "5":

    break
```

```
else:

    print("Invalid choice")
```

```
1 SA = {}
2
3 while True:
4     print("\n1.Create 2.Lookup 3.Delete 4.Display 5.Exit")
5     choice = input("Enter choice: ")
6
7     if choice == "1":
8         spi = input("Enter SPI: ")
9         src = input("Enter source IP: ")
10        dst = input("Enter destination IP: ")
11        SA[spi] = (src, dst)
12        print("Security Association created")
13
14    elif choice == "2":
15        spi = input("Enter SPI: ")
16        if spi in SA:
17            print("SA found:", SA[spi])
18        else:
19            print("SA not found")
20
21    elif choice == "3":
22        spi = input("Enter SPI: ")
23        if spi in SA:
24            del SA[spi]
25            print("Security Association deleted")
26        else:
27            print("SA not found")
28
29    elif choice == "4":
30        spi = input("Enter SPI: ")
31        if spi in SA:
32            print("SA found:", SA[spi])
33        else:
34            print("SA not found")
35
36    elif choice == "5":
37        break
38
39 print("Program ended")
```

Output

```
1.Create 2.Lookup 3.Delete 4.Display 5.Exit
Enter choice: 1
Enter SPI: 1001
Enter source IP: 192.168.1.1
Enter destination IP: 192.168.1.2
Security Association created

1.Create 2.Lookup 3.Delete 4.Display 5.Exit
Enter choice: 4
Security Associations: {'1001': ('192.168.1.1', '192.168.1.2')}

1.Create 2.Lookup 3.Delete 4.Display 5.Exit
Enter choice: 5

=== Code Execution Successful ===
```

4. Simplified PGP – Encryption and Digital Signature

The program encrypts an email message, generates a **digital signature**, decrypts the message, and verifies the signature.

```
import hashlib
```

```
p = 61
```

```
q = 53
```

```
n = p * q
```

```
phi = (p - 1) * (q - 1)
```

```
e = 17
```

```
d = 2753
```

```
message = input("Enter email message: ")
```

```
signature = pow(int(hashlib.sha256(message.encode()).hexdigest(), 16), d, n)
```

```
cipher = [pow(ord(c), e, n) for c in message]
```

```
print("Encrypted message:", cipher)
```

```
print("Digital signature:", signature)
```

```
plain = ''.join(chr(pow(c, d, n)) for c in cipher)
```

```
check = pow(signature, e, n)
```

```
original_hash = int(hashlib.sha256(message.encode()).hexdigest(), 16) % n
```

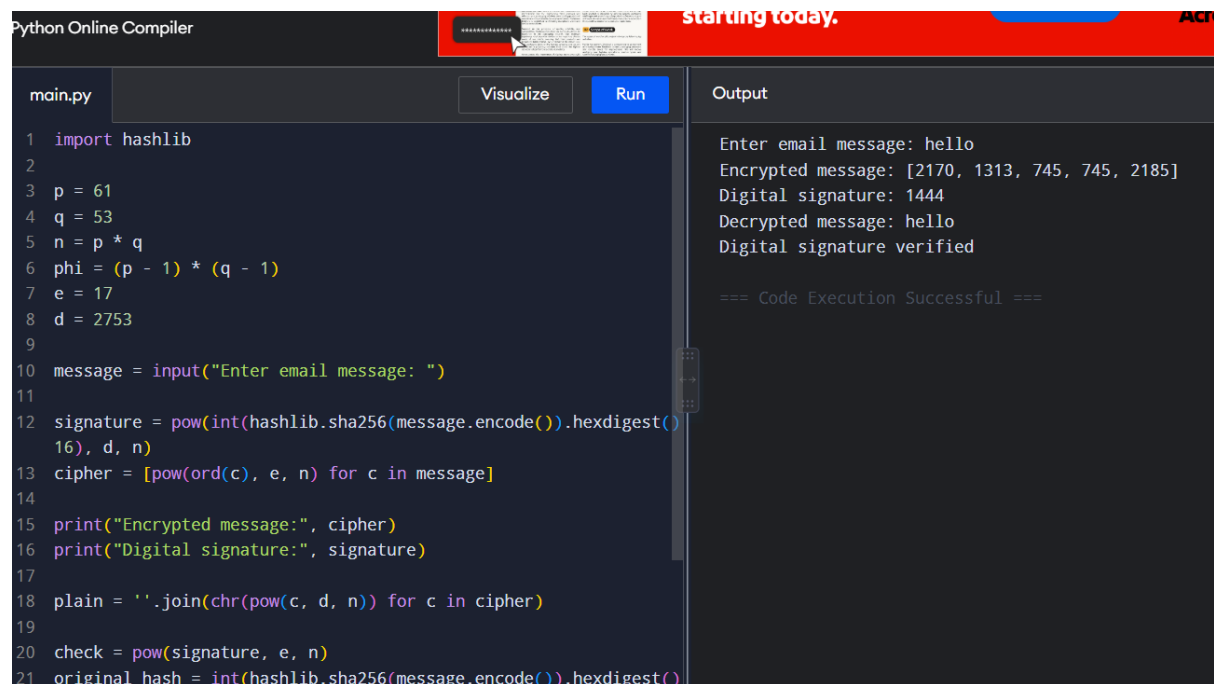
```
print("Decrypted message:", plain)
```

```
if check == original_hash:
```

```
    print("Digital signature verified")
```

```
else:
```

```
    print("Signature verification failed")
```



The screenshot shows a web-based Python IDE. The top bar includes the text "Python Online Compiler" and a red banner with "starting today." and "ACT". Below the bar, there are tabs for "main.py", "Visualize", and "Run". The "main.py" tab is active, displaying the following code:

```
1 import hashlib
2
3 p = 61
4 q = 53
5 n = p * q
6 phi = (p - 1) * (q - 1)
7 e = 17
8 d = 2753
9
10 message = input("Enter email message: ")
11
12 signature = pow(int(hashlib.sha256(message.encode()).hexdigest(),
13                    16), d, n)
14
15 cipher = [pow(ord(c), e, n) for c in message]
16
17
18 print("Encrypted message:", cipher)
19 print("Digital signature:", signature)
20
21
22 plain = ''.join(chr(pow(c, d, n)) for c in cipher)
23
24 check = pow(signature, e, n)
25 original_hash = int(hashlib.sha256(message.encode()).hexdigest(), 16) % n
26
27 print("Decrypted message:", plain)
28
29 if check == original_hash:
30     print("Digital signature verified")
31 else:
32     print("Signature verification failed")
```

The "Output" panel on the right shows the execution results:

```
Enter email message: hello
Encrypted message: [2170, 1313, 745, 745, 2185]
Digital signature: 1444
Decrypted message: hello
Digital signature verified

=== Code Execution Successful ===
```


5. Spam Email Classification

1. The program checks the email for suspicious words such as **free, offer, winner, prize, and urgent**. If enough suspicious words are found, it classifies the email as **SPAM**; otherwise, it is **LEGITIMATE**.

```
spam_words = [  
    "win", "winner", "free", "offer", "prize",  
    "money", "urgent", "click", "lottery", "discount"  
]
```

```
sender = input("Enter sender email: ")
```

```
subject = input("Enter subject: ")
```

```
body = input("Enter email content: ")
```

```
text = (sender + " " + subject + " " + body).lower()
```

```
score = 0
```

```
for word in spam_words:
```

```
    if word in text:
```

```
        score += 1
```

```
if score >= 2:
```

```
    print("Email Classification: SPAM")
```

```
else:
```

```
    print("Email Classification: LEGITIMATE")
```

main.py

VisualizeRun

```
1 spam_words = [
2     "win", "winner", "free", "offer", "prize",
3     "money", "urgent", "click", "lottery", "discount"
4 ]
5
6 sender = input("Enter sender email: ")
7 subject = input("Enter subject: ")
8 body = input("Enter email content: ")
9
10 text = (sender + " " + subject + " " + body).lower()
11
12 score = 0
13
14 for word in spam_words:
15     if word in text:
16         score += 1
17
18 if score >= 2:
19     print("Email Classification: SPAM")
20 else:
21     print("Email Classification: LEGITIMATE")
```

Output

```
Enter sender email: unknown@gmail.com
Enter subject: You won a free prize
Enter email content: Click now to claim your money
Email Classification: SPAM

=== Code Execution Successful ===
```